Timed Alignments

Thomas Chatain
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

Neha Rino
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

***Abstract*—The subject of this paper is to study conformance checking for timed models, that is, process models that consider both the sequence of events in a process as well as the timestamps at which each event is recorded. Time-aware process mining is a growing subfield of research, and as tools that seek to discover timing related properties in processes develop, so to does the need for conformance checking techniques that can tackle time constraints and provide insightful quality measures for time-aware process models. In particular, one of the most useful conformance artefacts is the alignment, that is, finding the minimal changes necessary to correct a new observation to conform to a process model. From alignments, one can derive measures of fitness, precision, and generalisation of a process model, which are crucial to analysing its performance. In this paper, we set our problem of timed alignment and solve two cases, corresponding to two metrics by which to compare timed processes, and seek to solve the alignment problem, that is, the problem of calculating the closest trace in the language of a timed model to a particular observed event trace. For the first metric, we have an algorithm that aligns traces to any extended free choice time petri net, with time complexity linear in the size of the model and it's transition set. For the second metric, we have a general approach to solve the alignment problem using the simplex algorithm, which has polynomial average time complexity and exponential worst case time complexity, and for a subclass of models we have a quadratic time algorithm.**

***Index Terms*—Conformance checking, Alignments, Timestamps, Timed Petri nets**

I. Introduction

A. Conformance Checking and Alignments

Process mining studies vast systems through their event logs, and seeks to extract meaningful ways to model the underlying patterns or processes that govern the behaviour of the system, sometimes in order to better understand the system, or as is often the case, predict future behaviour [1]. Once such a process model is obtained, through machine learning or other related techniques, it is natural to ask how one is sure the obtained model is a reasonable approximation of the system's behaviour at all, especially given the lack of explainability in the blackbox approach ML takes to producing solutions. This is where conformance checking comes into the picture, as it is the art of judging the performance of a process model by comparing its behaviour to the observed behaviour of the system, that is, relating modelled and recorded behaviour of a process to each other, independent of the origin of the model [2]. Observed behaviour usually comes in the form the traces of an event log, as a sequence of events that often belong to multiple distinct interleaved instances of processes of the system. On the other hand, we have process models, that were either obtained automatically through mining, or were constructed manually, and act a blueprint to describe what the underlying processes of any given system are supposed to look like. Measures of how well a model reflects system behaviour include fitness, precision, generalisation, and simplicity. We often do not want the system to always precisely generate any and all possible future system behaviour as this can give trivial models accepting for example all execution strings, but on the other hand, neither should they simply regurgitate the event log and accept no new system behaviours, a phenomenon known as overfitting, or not generalising properly. What is often much more useful is a process model that can, up to some small error factor, approximate any reasonable future system behaviour quite closely. This flexible boundary between what is expected and what is possible is precisely what makes a model useful.

In Aryan Adriansyah's seminal thesis [3] , we obtain the notion of an alignment, that is, the minimal series of corrections needed to transform an observed event trace into the execution of the process model that most closely mimics it. It is often seen as an execution of the synchronised product of the process model and a *trace model* ie a simple model that captures exactly one event log trace. This gives us a series of edits, usually insertions or deletions, that can transform a model run into the observed run. For our purposes we will use the term alignment to mean the optimal run of the model that aligns closest to the observation, and how to transform this optimal execution to the observed one will be addressed separately as a matter of distance computation over a set of edit moves. Alignments thereby help pinpoint exactly where inevitable deviations from expected behaviour occur, and the more distant the aligning word of a model is to its observed trace, the worse the model is at reflecting real life behaviour. This can either be used as a tool to help repair the process model to

more accurately reflect the observation, or, it can be used to repair the log, or even issue feedback on the behaviour of the underlying system to help it conform to the model better, depending on which components of the operation are the most trustworthy.

B. Time-aware Processes

Process models are often formalised using the notion of a Petri net, a standard object used to describe distributed systems. They both offer a graphical means by which to represent a complex system, and a formal semantics for their execution, which allows one to mathematically analyse the same. They are useful when trying to express dependencies in seemingly parallel parts of the system, the graph structure allowing for branching (i.e. choice), convergence (i.e. synchronisation), and cycles (i.e iteration) in system behaviour. Assuming the event logs are a list of words over a finite alphabet (the set of possible discrete events), the problem of calculating the alignment has been extensively studied [3] [4]. The notion of distance used on these words used is usually either Hamming distance or Levenshtein's edit distance. Given that Petri net processes already do have certain temporal characteristics, such as a (partial) sense of what happens before and after, and a notion of causal dependency between events, it is natural to want to study explicitly timed systems, that is, systems that not only retain the sequence of events that occur, but the timestamps that show when each event happens. By considering events along with their timestamps when mining processes, observations regarding complex time constraints can be obtained, such as the minimum delay between two events, or the maximum duration the system takes to converge upon a state, or meeting deadlines, all of which are highly relevant in real world applications [5] [6] [7]. In addition, one may want to predict the timestamps of processes [8]. In the community, there are ways to use existing process model notation in order to denote some time constraints. BPMN 2.0 comes equipped with *timer events*, which may be catching or interrupting in nature, and can record absolute, relative, and cyclical time constraints. For our purposes, we use a slightly different formal object whose semantics are capable of expressing both concurrency properties, and the relative and cyclical time constraints that BPMN equipped with timers can. Here, we mean time Petri nets, that is, Petri nets augmented with the ability to record and check the duration it takes to fire a transition once enabled, using which one can impose certain constraints on the relationships between the timestamps of different events.

So far in the study of the alignment problem for process models, we find one notable gap. The process models for which alignment is studied do not consider event timestamps, and constraints imposed upon them. Time-aware process mining seeks to study both what sort of underlying processes govern system behaviour, and what sort of time constraints they can impose on when certain events can occur [9] [10] [11]. The need for such a setting is clear, as specifying and verifying temporal properties, and concrete time constraints regarding the durations of events can provide critical information about a system's behaviour. As time-aware process mining grows popular, new quality measures and conformance checking techniques must be developed that are sensitive to temporal constraints. For this, crucially, one needs to define distance functions that can meaningfully compare and separate different time processes. This paper seeks to provide a framework by which to do the very same.

C. Three Distances, and Algorithms to Align them

In this paper, we propose three different distance functions over timed words, corresponding to only editing one point in a word at a time (stamp edits), forcing every edit in a word to ripple out and displace all of its causal successors (delay edits), or a combination of the two (mixed edits). The first is essentially Manhattan distance, and hence is extremely practical, and has a quadratic time algorithm in the case of a structurally restricted class of models (linear causal processes), that is, process models whose runs look like straight lines with no branching or concurrency. For all other types of models, we present an encoding in simplex that solves the problem, albeit with exponential worst-time complexity and good efficiency in practice. The second distance function utilises the structure of the process model to study distances between time processes, and we present a straightforward algorithm for solving the alignment in this case, whose time complexity is linear in the size of the causal process and the transition set of the model. This setting may be of slightly limited scope as it is only relevant if one always knows that every single error will have knock-on effects, such as in event logs that only perceive relative durations but output absolute timestamps. The third is the most versatile, and the most interesting for our purposes. In this setting, edits can either be stamp moves or delay moves, allowing one to choose whether to let an edit stay local or affect the rest of the process, depending on which would be cheaper. We present a linear time algorithm for solving the alignment problem in this setting, but once again only for process models with linear causal processes as in the first setting.

II. Preliminaries

A. Timed Process Models

We represent events as timed letters, that is of the form (a, t) where $a \in \Sigma$ is the name of the action taken during an event, and t denotes the time at which said action was taken. An event trace is then a sequence of events of the form $\gamma \in (\Sigma \times \mathbb{R}^+)^*$, seen as a timed word. For the context of this paper, we will often ignore the untimed parts of timed words, that is, the projection onto Σ^*, leaving just

the time-series of the timed words, a sequence belonging to $\mathbb{R}^{+*}$.

The timed process model that attempts to capture the behaviour of all observed event traces recorded from the system is represented here as a labelled time Petri net.

Definition 1 (Time Petri Net)**.** *A labelled time Petri net is a tuple $N = (P, T, F, SI, \lambda, M_0, M_f)$, where*

1) *P is a set of places*
2) *T is a set of transitions*
3) *$P \cap T = \emptyset$*
4) *$F \subseteq (P \times T) \cup (T \times P)$ is a flow relation*
5) *$SI : T \to \mathbb{I}$ is the static interval function, where $SI(t) = (Eft(t), Lft(t))$ such that Eft stands for earliest firing time, and Lft for latest firing time.*
6) *$\lambda : T \to A$ is the labelling function, labelling transitions with actions*
7) *$M_0 : P \to \mathbb{N}$ is the initial marking of N*
8) *$M_f : P \to \mathbb{N}$ is the final marking of N*

The semantics of a time Petri net are intuitive, if a bit involved. Given a transition $t \in T$ we define the pre-set of t as ${}^\bullet t = \{p \in P | (p, t) \in F\}$ and its post-set similarly is defined as $t^\bullet = \{p \in P | (t, p) \in F\}$, just as it is defined for untimed Petri nets (the presets and post-sets of places are defined similarly). A transition t of a time Petri net is *enabled* at marking M iff $\forall p \in {}^\bullet t : M(p) > 0$. The set of all enabled transitions at a marking M is denoted by *Enabled(M)*.

This formalism begins to differ from untimed Petri nets with the notion of states and clock functions. The current marking is not sufficient to define the state of the whole system, as the time elapsed since a transition was enabled is crucial information to define the execution of the system. Hence, a *state* of a time Petri net $N = (P, T, F, SI, M_0)$ is a pair $S = (M, I)$, where M is a marking of N and $I : Enabled(M) \to \mathbb{R}^+$ is called the *clock function*. The initial state is therefore $(M_0, \mathbf{0})$, where $\mathbf{0}$ is the zero function.

A transition t is fireable from state $S = (M, I)$ after delay $\theta \in \mathbb{T}$ iff t is enabled at M, and updating the clock function to increment by delay θ will keep t's new clock value $I(t) + \theta$ in the range $[Eft(t), Lft(t)]$ as determined by the static interval function. This is also referred to as t firing at time $I(t) + \theta$ at state (M, I).

Once t is fired, the marking is updated to M' as in the untimed case (subtracting one token from each preplace, and adding one token to each postplace), and the clock function I is updated to I' by increasing clock values by θ for any transition that was enabled at both M and M', setting clock values to 0 for any transitions newly enabled by M' that were not enabled at M, and leaving clock values undefined for any transitions that are not currently enabled. This is denoted by $(M, I)[t\rangle(M', I')$

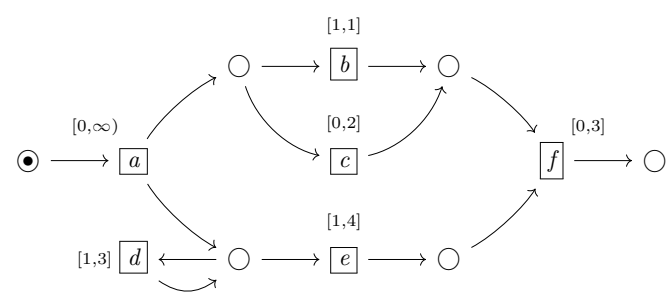
An important feature of time Petri nets is the notion of urgency, that is, if a transition has been enabled then it must either be disabled (lose some tokens from its preset due to another transition firing) or be fired before the latest firing time lapses. It cannot be allowed to simply *time-out*, i.e, waiting, unfired, throughout the period of time is it capable of firing, and then being disabled just by having the Lft lapse, that is if t is enabled at marking M and has clock value $Lft(t)$, it must fire, or another transition must fire at the same instant disabling t.

Now, we want a notion of a valid execution of the model, that is, valid instances of processes the model wishes to recognise. This is why we equip our time Petri nets with a notion of a final state, or in this case, a final marking. A valid execution begins at the initial marking, and fires a sequence of transitions (representing a series of activities occuring at certain times) and the updated marking at the end of the firing sequence being M_f, with any clock function I.

Definition 2 (Language of a time Petri net)**.** *A word $w = (a_0, a_1, \dots a_n) \in A*$ is in the language of the labelled time Petri net $\mathcal{L}(N)$ if there is a firable sequence of transitions $(t_0, t_1 \dots t_n) \in T^*$ such that $\lambda((t_0, t_1, \dots, t_n)) = w$ and they transform the initial marking into the final one, that is, for some clock function I on M_f,*

$$(M_0, \mathbf{0})[t_0, t_1, \dots t_n\rangle(M_f, I)$$

Example 1. *Consider the following example of a time Petri net N:*



One possible execution of N would be for the firing sequence

$$w = (a, 1)(b, 2)(d, 3)(e, 4)(f, 5)$$

The initial marking only has a enabled, and firing a at time 1 updates the marking by removing the token from a's preplace and filling its two post-places, thereby enabling b, c, d, e. Now, note by urgency that c can never fire past time 2 as then b would have already fired by urgency, but in this execution this doesn't matter as b fires at 2 regardless, filling one preplace of f. Then d fires, having been enabled for 2 units of time, thereby resetting the enabling time of both d and e to zero, so that when e does fire it has only been most recently enabled for 1 unit of time. Now, finally, f is enabled, and fires after 1 unit of having been enabled.

Now we will want to build some vocabulary to help us talk about individual executions of timed words on time Petri nets. These correspond to runs in nondeterministic automata, but in this setting every configuration is a set of

token-filled places, so to represent a particular execution we use what we define below as causal processes.

Definition 3 (Causal Net)**.** *A causal net $CN = (B, E, G)$ is a finitary, acyclic net where*

$$\forall b \in B : |b^\bullet| \leq 1 \wedge |{}^\bullet b| \leq 1.$$

This is the ground upon which a process of the time Petri net is played. It can be viewed as the original Petri net itself, but every time a place is revisited, it is copied afresh to ensure that the execution only ever moves forward. Hence, multiple elements of B or E map to the same element in P or T respectively, a notion that is expressed via the map p defined below.

Definition 4 (Homomorphism)**.** *Let N be a time Petri net with place set P and transition set T, and $CN = (B, E, G)$ be a causal net. A mapping $p : B \cup E \to P \cup T$ is a homomorphism if*

1) $p(B) \subseteq P$
2) $p(E) \subseteq T$
3) *$\forall e \in E$ the restriction of p to ${}^\bullet e$ is a bijection between ${}^\bullet e$ and ${}^\bullet p(e)$ and the restriction of p to $e^\bullet$ is a bijection between $e^\bullet$ and $p(e)^\bullet$ and the restriction of p to $Min(CN)$ is a bijection between $Min(CN)$ and M_0.*

Definition 5 (Causal Process)**.** *A causal process of a TPN is a pair (CN, p) where CN is a causal net and p is a homomorphism from CN to TPN.*

Using p, elements of CN are identified with their corresponding *originals* in the time Petri net, and as a result, any causal process corresponds uniquely to an untimed run on the untimed version of a given time Petri net.

Definition 6 (Timing Function)**.** *A timing function $\tau : E \to \mathbb{T}$ is a function from events of a causal process into time values.*

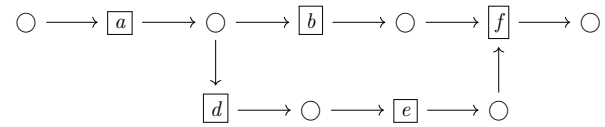
B. Linear Causal Processes, a relevant class

We take a moment to discuss a structurally restricted class of processes, that is, causal processes whose graphical structure is that of a straight line. These reflect the executions of time Petri nets that do not have any branching points ($e \in E : |e^\bullet| > 1$) and naturally as a consequence, no points of synchrony ($e \in E : |{}^\bullet e| > 1$). This essentially reflects the executions of automata, allowing for exclusive branching ($p \in P : |p^\bullet| > 1$) and iteration (that is, cycles in the model) only. It loses the ability to capture properties of a concurrent nature, as the run never splits into more than one token. On the other hand, studying the language of this restricted class and the alignment problem over it becomes substantially simpler, the G preorder becomes total, and so to begin with, this is a significantly more tractable class of time processes.

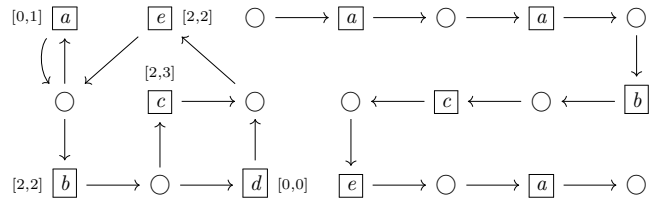
Example 2. *Going back to the net N in example 1, and the firing sequence we analysed then*

$$w = (a, 1)(b, 2)(d, 3)(e, 4)(f, 5)$$

We now construct the causal process of this execution, and see that it is indeed branching, as the very first transition, a, itself has two post-places, which violates the no branching condition $e \in E : |e^\bullet| \leq 1$.



Example 3. *On the other hand, consider the following time Petri net N_1 and note that while it has branch points and joins and loops, they're all at places :*



Now, as all the branching and joining happens at places rather than transitions, its executions all have linear causal processes, such as the above causal process for the untimed word $w = aabcea$.

This causal process is unrolled from the Petri net, exactly the way runs of finite state automata are simple paths over the graph of the automaton. We henceforth assume that wherever needed, such a causal process can be obtained efficiently.

III. The Alignment Problem in Timed Settings

The alignment problem, given a trace from an event log and a process model, involves finding a valid execution of the process model that is closest to the trace under some metric.

Definition 7 (The General Alignment Problem)**.** *Given a process model N denoted by a Time Petri Net and a timed trace σ we wish to find a timed word $\gamma \in \mathcal{L}(N)$ such that*

$$d(\sigma, \gamma) = \min_{x \in \mathcal{L}(N)} d(\sigma, x)$$

for some distance function d on timed words.

In the untimed setting, this is viewed as a problem of minimizing cost over a series of edit moves, either insertions (where the model reads a letter that wasn't in σ) or deletions (where σ has a letter that the model does not wish to read). When aligning timed words, clearly there are two crucial aspects to the problem, which are, how one deals with deviations in the action labels themselves (the untimed letters), versus how one deals with deviations in just the timing properties of the word (the timestamps). The alignment problem has been extensively studied for the untimed case, and so, if we simply sought to align the action labels of the word to the underlying untimed

Petri net, this is a solved problem. On the other hand, timestamps are attached to the letters they belong to, so in a sense, performing task A at time t_1 as compared to performing task B at time t_2 is not easily comparable, as the time constraints on different tasks are often different. In this manner, it is not clear how meaningful it would be to define edit moves that transform timestamps that are attached to different tasks.

This is why, we first restrict our attention to the case where the untimed part of σ does match the model, that is to say, if $\sigma = (w, \tau)$ where w is the untimed letter sequence and τ is the timestamp sequence. then we assume that w has a valid causal process (CN, p) over N. Clearly this degenerate case of the problem needs to be solved if the general timed alignment problem is to be solved, and we proceed to show that this case is interesting and complex in its own right.

This ensures that we can compare the timing constraints on analogous parts of the process, and hence reason meaningfully about deviances from the model that are purely based on the timing of the process. This gives us a slightly different problem than the general setting of the alignment problem, that focuses purely on timing properties, that is, a purely timed alignment problem, as defined below :

Definition 8 (The Purely Timed Alignment Problem)**.** *Given a process model N denoted by a Time Petri Net and a timed trace (w, σ), and a valid causal process of the untimed part w, we wish to find a valid timing function γ such that*

$$d(\sigma, \gamma) = \min_{x \in \mathcal{L}(N)} d(\sigma, x)$$

for some distance function d on timing functions.

Hence, for the first few sections of this paper, we will proceed to ignore the untimed parts of the words, i.e we assume the word σ we wish to align comes with a valid causal process (CN, p) of the model, and simply seek to find timing functions on said causal process that are both valid, and minimize distance to σ under the metric of choice.

We shall revisit the general timed alignment problem in section VI, and show how our methods for solving the purely timed alignment problem can be adapted using existing techniques to provide approaches for solving the general timed alignment problem.

A. Edit Moves on Timing Functions

Now we come to the problem of deciding how to compare two timing functions over the same causal process, and quantify how close they are to each other.

Much like Levenshtein's edit distance, popularly used in the untimed case of the alignment problem, we view the definition of these distances as an exercise in cost minimisation over the set of all transformations between two words. In order to formalise the same, we need to define what the valid moves of such a transformation could be. We define *moves* as functions that map one timing function over (CN, p) to another. What sort of functions are useful notions of transformation on a timed system?

Example 4. *Let us go back to example 1, and study the process model N presented there. Now, say we had the following words that did not fit the model, and we wished to analyse how best to modify them to fit them back into the model.*

We start with $(u_1, \sigma_1) = (abdef)(1, 3, 4, 6, 6)$. A reasonable candidate for the closest valid execution would be if the timing function σ_1 were replaced by $\gamma = (1, 2, 4, 6, 6)$, and it feels reasonable to say that the cost for aligning σ_1 to this γ is 1. A way to arrive at this conclusion is by trying to execute σ's firing sequence, where we see that only the guard for b fails. If b were just shifted to fire at 2 instead, the whole run would execute without a hitch. This sort of local, almost typographical error can often happen in systems, and it is the simplest kind to fix.

We start with $(u_1, \sigma_2) = (abdef)(1, 2, 5, 9, 12)$. The closest valid timing function would try to preserve the positions of a and b but the moment it tries to fire d, σ_2 runs late throughout. The closest we can get, intuitively, is to fire every transition in the lower branch as late as possible, giving $\gamma = (1, 2, 4, 8, 11)$. Now, when trying to compare these firing sequences, we can view it just like we did for σ_1, as d, e and f all fire later than they should, each timestamp is moved back once, giving an aligning cost of 3. There is however, another way to see this deviation. This cascading chain of errors can be fixed if d is moved back to fire at 4, and all the relative relationships between d and its successors are preserved. This views the tasks e and f as only caring about when d ended, which makes sense, because they depend on d's completion. This means, the switch from the timestamp series $(5, 9, 12)$ to $(4, 8, 11)$ can be viewed as only a cost 1 edit, if we assume e and f only care about the relative durations or delays between activities. This is a slightly more complex error to conceive of, but it reflects the fact that if a delay at the beginning caused the whole process to occur too late, the important thing to fix is just the delay at the beginning, and the rest of the process will now conform to the model as needed.

Based on the above example, we naturally arrive at two types of moves.

We define a *stamp move* as a move that translates the timing function only at a point, ie, that edits a particular element of the timestamp series τ. Calculating d_t between two traces can be viewed as a cost minimization process in aligning the traces using only stamp moves.

Definition 9 (Stamp Move)**.** *Given a timing function $\gamma : E \to \mathbb{R}$, formally, we define this as :*

$\forall x \in \mathbb{R}, e \in E : stamp(\gamma, x, e) = \gamma'$ *where*

$$\forall e' \in E : \gamma'(e') = \begin{cases} \gamma(e') + x & e' = e \\ \gamma(e') & otherwise \end{cases}$$

The next type of move we describe is the more novel and interesting *delay move*. By formulating this type of edit move, we sought to leverage the structure of the process model itself, by reflecting the causal relationships the pre-order G causes. If an event is G-reachable from another, that means the first event is strictly in the causal history of the second so changing the timestamp of such a causal predecessor has consequences for all of its causal descendents, while leaving any causally unrelated (i.e, non-G-reachable) events undisturbed.

In other words, a delay move at e will preserve relative relationships between timestamps in the future, at the cost of shifting the timestamp of every causal descendent of e by the same amount. This gives us the following definition :

Definition 10 (Delay Move)**.** *Given a timing function $\gamma : E \to \mathbb{R}^n$, we define a delay move applied to it as follows :*
$\forall x \in \mathbb{R}, e \in E : delay(\gamma, x, e) = \gamma'$ *where*

$$\forall e' \in E : \gamma'(e') = \begin{cases} \gamma(e') + x & e' \geq_G e \\ \gamma(e') & otherwise \end{cases}$$

Now, armed with these types of moves, three natural notions of distance can be constructed.

Definition 11 (Stamp Only Distance : d_t)**.** *Given any two timing functions τ_1, τ_2 over the same causal process (CN, p), we define the stamp-only distance d_t as follows :*

$$d_t(\tau_1, \tau_2) = \min\{cost(m) | m \in Stamp^*, m(\tau_1) = \tau_2\}$$

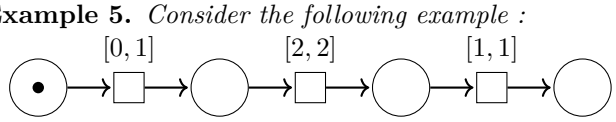
Definition 12 (Delay Only Distance : d_θ)**.** *Given any two timing functions τ_1, τ_2 over the same causal process (CN, p), we define the delay-only distance d_θ as follows :*

$$d_t(\tau_1, \tau_2) = \min\{cost(m) | m \in Delay^*, m(\tau_1) = \tau_2\}$$

Definition 13 (Mixed Moves Distance : d_N)**.** *And thirdly, given any two timing functions τ_1, τ_2 over the same causal process (CN, p), we define the mixed move distance d_N as follows :*

$$d_t(\tau_1, \tau_2) = \min\{cost(m) | m \in (Stamp \cup Delay)^*, m(\tau_1) = \tau_2\}$$

Example 5. *Consider the following example :*



Now for this N, let the observed trace $\sigma = (3, 4, 5) \notin \mathcal{L}(N)$.

The best d_t alignment for the example in the diagram below is $\gamma = (1, 3, 4)$ with minimum cost $d_t(\sigma, \gamma) = 4$.

The best $d_t = \theta$ alignment for the example in the diagram below is also $\gamma = (1, 3, 4)$, but this time with minimum cost $d_\theta(\sigma, \gamma) = 3$, evidenced by the move sequence $(delay(-2, 1)delay(+1, 2))$.

And lastly, the best d_N alignment for the example in the diagram below is also $\gamma = (1, 3, 4)$, the sequence of moves being simply one stamp and one delay move at the start, $m = stamp(-1, 1)delay(-1, 1)$, and now with minimum cost $d_N(\sigma, \gamma) = 2 < \min\{d_t(\sigma, \gamma), d_\theta(\sigma, \gamma)\}$.

IV. Results and Algorithm : Delays Only Setting

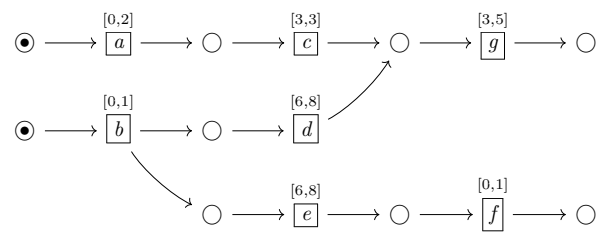
With delay edits, which constitute a new way of thinking about how a time-series can be transformed, comes a new perspective with which we can view timing functions over causal processes. There is of course the standard definition, $\tau : E \to \mathbb{R}^+$ that assigns to each event a timestamp that records exactly when the event occurs.

Instead, thinking along the lines of delays and durations between events occuring, we will often benefit from considering the following representation when speaking about delay moves, and so we define a way by which to view a timed word not in terms of its absolute timestamps, but by the delays between relevant timestamps.

Definition 14 (Flow Function)**.** *Given a causal process (CN, p) over N, where $CN = (B, E, G)$, and a (not necessarily valid) timing function $\tau : E \to \mathbb{R}^+$, we first define the flow function of τ, $f_\tau : E \to \mathbb{R}^+$ such that*

$$f_\tau(e) = \begin{cases} \tau(e) & {}^{\bullet\bullet}e = \emptyset \\ \tau(e) - \tau(e') & e' \in {}^{\bullet\bullet}e, \\ & \tau(e') = \max\limits_{e'' \in {}^{\bullet\bullet}e} \{\tau(e'')\} \cup \{0\} \end{cases}$$

Example 6. *Consider the following process model N_2:*



The word

$$w = (a, 1)(b, 2)(c, 5)(d, 6)(e, 6)(f, 8)(g, 8)$$

is not a member of $\mathcal{L}(N_2)$ but its untimed part belongs to the language of the untimed underlying net, so it does have a valid causal process. For this net and word, we can confuse transition labels with event labels, so we can define its flow function as

$$a \mapsto 1, b \mapsto 2, c \mapsto 4, d \mapsto 4, e \mapsto 4, f \mapsto 2, g \mapsto 2$$

Where the order of events is clear (such as here, where it is alphabetical, or in linear causal processes) we can write the flow function as a tuple, as follows :

$$f_w = (1, 2, 4, 4, 2, 2)$$

Note that e' as defined here is the latest causal predecessor of e, that is, it is the firing of e' that enabled e. Hence, f_τ as defined produces exactly the time durations that the guards of each transition in the model checks, ie the clock function values during the run.

As defined, we see that if a word is in the language of the model, then its f function maps events to values that lie within the constraint that the event's corresponding

transition demands, that is, it is perfectly aligned with the model with distance 0. This condition is unfortunately not sufficient though, due to urgency concerns, but it is quite close to the exact condition necessary for a word to be in the language of a time Petri net.

Also note that given the underlying causal process and the resulting f_τ, we can reconstruct τ quite straightforwardly as

$$\forall e \in E : \tau(e) = \sum_{e' \leq_G e} f_\tau(e')$$

Lemma 1 (Delay only distance : d_θ)**.** *d_θ as defined above based on moves, is equivalent to the following formulation directly using the flow function previously defined, as follows :*

$$d_\theta(\tau_1, \tau_2) = \sum_{e \in E} |f_{\tau_1}(e) - f_{\tau_2}(e)|$$

Proof. By noting that delay moves on a trace τ translate exactly to stamp moves on f_τ, we can deduce by the same argument as Lemma 3 that the distance function d_θ can be seen to be identical the manhattan distance on flow functions, i.e, the above formulation. □

The flow function thereby is a dual representation of timing functions, much like the relationship between graphs and line graphs, as if a timing function traditionally labels its transition nodes with timestamps, the flow function labels edges leading up to transitions with the duration since said transition was enabled.

Example 7. *Let us study the process model N_2 from example 6 once again : Say we want to align*

$$w = (a,1)(b,2)(c,5)(d,6)(e,6)(f,8)(g,8)$$

to N_1. We recall that we calculated its flow function to be

$$a \mapsto 1, b \mapsto 2, c \mapsto 4, d \mapsto 4, e \mapsto 4, f \mapsto 2, g \mapsto 2$$

As membership in the language only depends on the flow values, we try to pick the best flow component for each letter, that is, flow components that are as close to w's as possible while keeping the alignment in the language. Choosing naively like this gives us the flow function

$$a \mapsto 1, b \mapsto 1, c \mapsto 3, d \mapsto 6, e \mapsto 8, f \mapsto 1, g \mapsto 3$$

From which we can reconstruct the closest word

$$u = (a,1)(b,1)(c,4)(d,7)(e,7)(f,8)(g,10)$$

Clearly if one tries to move any component of u's flow function any closer to that of w, the word will no longer be in the language.

We could reason the way we did in the example above because the delay only distance, d_θ, is built so as to allow local edits that do not affect global membership in the language of the time Petri net, as both the time Petri net and delay edits concern themselves only with the duration of time elapsed between a transition's enabling and its firing. This relationship is especially clear in extended free choice time Petri nets, where the only events that determine the fireability of an event are its causal history and events with the exact same preset, which would naturally disable it by firing. Hence, we present the following algorithm, which minimizes the delay cost at each transition locally, while still overall ensuring membership in the language of the model.

Algorithm 1 Local d_θ Algorithm

Input : N, $CN = (B, E, G), p$, $\sigma : E \to \mathbb{R}^+$
Output : $\gamma : E \to \mathbb{R}^+$ such that γ is a valid timing function and $d_\theta(\gamma, \sigma) \leq d_\theta(x, \sigma)$ for all valid timing functions x
$Curr = Min(CN)$
$Enabled = \{t | {}^\bullet t \subseteq p(Curr)\}$
$FD = \{(t, Eft(t), l, 0) | t \in Enabled, l = \min_{\bullet t' = \bullet t} Lft(t')\}$
$Soon = \{(t, eft, l, toe) \in FD | l + toe$ is minimal in $FD\}$
while $E \neq \emptyset$ **do**
  Pick $(t, eft, l, toe) \in Soon, t \in p(E)$
  $E \leftarrow E \setminus \{e | p(e) = t\}$
  $e \leftarrow p^{-1}(t)$
  $Curr \leftarrow \{Curr \setminus \{{}^\bullet e\}\} \cup \{e^\bullet\}$
  $f_\gamma(e) = \text{argmin}_{x \in [eft, l]} |x - f_\sigma(e)|$
  for all $t' \in T \wedge {}^\bullet t' = {}^\bullet t$ **do**
    $Enabled \leftarrow Enabled \setminus \{t'\}$
    $FD \leftarrow FD \setminus \{(t', e', l', toe')\}$
  end for
  for all $t' \in T \wedge t'^\bullet = t^\bullet$ **do**
    $Enabled \leftarrow Enabled \cup \{t'\}$
    $eft' \leftarrow Eft(t')$
    $l' \leftarrow \min_{\bullet t'' = \bullet t'} Lft(t'')$
    $toe' \leftarrow toe + f_\gamma(e)$
    $FD \leftarrow FD \cup \{(t', eft', l', toe')\}$
  end for
  $Soon = \{(t, eft, l, toe) \in FD | l + toe$ is minimal in $FD\}$
end while
return γ

Theorem 2. *Algorithm 1 is correct on extended free choice time petri nets, that is, its result γ has the properties :*

1) *(CN, p, γ) is a valid time process of N.*
2) *$d_\theta(\gamma, \sigma) \leq d_\theta(x, \sigma)$ for all x such that (CN, p, x) is a valid time process of N.*

Remark 1. *Note that Algorithm 1 runs through the events of the causal process E exactly once, and each time runs through the set of currently enabled transitions, and the total set of transitions, and hence its time complexity is linear in the size of the input $|CN|$, and the size of the transition set, i.e, $O(|E||T|)$.*

V. Results and Algorithm : Stamp Only Setting

Lemma 3 (Stamp only distance : d_t)**.** *d_t as defined above is equivalent to the Manhattan distance or taxicab distance between two points of $\mathbb{R}^n$, where $|E| = n$, say $\mathbf{t} = (t_1, t_2, \dots t_n)$ and $\mathbf{s} = (s_1, s_2, \dots s_n)$, defined as*

$$d_t(\mathbf{t}, \mathbf{s}) = \sum_{i=1}^{n} |t_i - s_i|$$

Proof. The way we see this is as follows. Given a causal process $CN = (B, E, G)$ and two timing functions (not necessarily constituting valid timings) τ_1, τ_2, let

$$d_t(\tau_1, \tau_2) = \sum_{e \in E} |\tau_1(e) - \tau_2(e)|$$

This is both achievable by stamp only moves by doing the corresponding $\tau_1(e) - \tau_2(e)$ stamp move to the second word, and is the least cost any such stamp only path can take as Manhattan distance obeys the triangle inequality, and every stamp move results in a word at exactly the same Manhattan distance from its predecessor as the cost of said stamp move.

□

A. General Bounded Time Petri Nets

The alignment problem in this case can be viewed as a problem of convex optimization, minimizing the stamp only cost, that is,

$$cost(\tau|_i) = \sum_{j=1}^{i} |\tau_j - \sigma_j|$$

over the set of possible timestamp series. What remains is to show that the space of valid runs of the model is a convex set, and this set has been studied extensively in [13]. In this paper, the concept of a firing domain is proposed, that is, at each configuration of the time Petri net, the set of valid time intervals in which a transition enabled at that configuration can fire. Using these firing domains, states are grouped into state classes, that is, sets of states with clock functions in the same firing domain, and for bounded time Petri nets there are finitely many such state classes. Calculating equivalence classes of the same, successively, till we find final configurations, gives a characterisation of the solution set of all valid timestamp series, and this is proved to be of the following form, which is a set of constraints defining a convex set.

Hence, we claim that using an efficient encoding of the target function using the firing domain formulation described above, the problem can be solved using the simplex algorithm, which is used to solve convex optimization problems in good time in practice, and having worst case exponential time complexity.

Theorem 4. *Given a bounded time Petri Net N, an observed trace σ, and its causal process $CN = (B, E, G)$, the alignment problem can be viewed as seeking the vector $(\gamma_1, \gamma_2, \dots \gamma_{|E|})$ that minimizes the quantity*

$$\sum_{e \in E} |\gamma_e - \sigma_e|$$

over the set $\gamma \in \mathcal{L}(N)$.

This can be cast as a convex optimization problem, i.e,

$$\underset{\alpha - \beta \in \mathcal{C}, \alpha, \beta \geq 0}{\text{minimize}} \quad \alpha + \beta$$

Where $\mathcal{C}$ is a convex set.

Remark 2. *By the above theorem we see that the alignment problem for d_t is easily cast as a simplex instance, and can be solved by any linear optimization solver. We can thereby solve the alignment problem in polynomial average running time, but this approach does have theoretically exponential worst time complexity. This bound is not necessarily tight, but either improving this bound or proving its tightness are left for future work. While the general case as posed seems to be hard to solve efficiently, there is a subclass of process models for which the problem is significantly more tractable, and moreover, has a quadratic time solution.*

B. Linear Causal Processes

We are given a process model N, an observed trace σ, and its underlying linear causal process $CN = (B, E, G)$ with homomorphism p for the untimed run of σ on N.

Suppose we tried to express the cost of aligning a prefix of σ to some timing function τ on the truncation of the linear causal process up to the ith transition, call it *cost*.

$$cost(\tau|_i) = \sum_{j=1}^{i} |\tau_j - \sigma_j|$$

Now, note the following equation holds for the $(i+1)$th static interval constraint $[a, b]$:

$$\min_{\tau|_{i+1} \in \mathcal{L}^{i+1}(N)} cost(\tau|_{i+1}) =$$

$$\min_{\tau_{i+1} - \tau_i \in [a,b]} (|\tau_{i+1} - \sigma_{i+1}| + \min_{\tau|_i \in \mathcal{L}^i(N)} cost(\tau|_i))$$

This suggests that the minimal cost of aligning prefixes σ to prefixes of the model can be recursively calculated as a function of the value of the last timestamp in the alignment of a given prefix.

$$g_i(d) = \min_{\tau|_i \in \mathcal{L}^i(N) \wedge \tau_i = d} cost(\tau|_i)$$

Now, the previous equation can be expressed in terms of the g functions as follows :

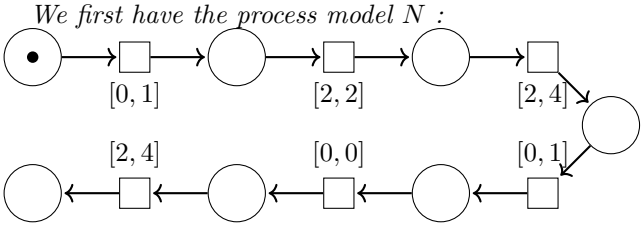
$$g_{i+1}(d) = |d - \sigma_{i+1}| + \min_{d - d' \in [a,b]} g_i(d')$$

If we can calculate the family of functions $\forall i \leq n : g_i$ efficiently, then the minimal cost for aligning σ to N is simply

$$\min_d g_n(d)$$

Example 8. *We take a moment to look at an example of the computation of this family of functions we proposed by considering the following d_t-alignment problem :*

We first have the process model N :



The observed trace is $\sigma = (1, 2, 4, 6, 6, 8)$.

Now, we begin to compute the family of functions g_i, starting with $g_1(x) = |1 - x|$ in the domain $[0, 1]$.

Now, utilising the recursion, we try to compute g_2.

The new domain is $[2, 3]$, a simple translation of the previous one because the guard on the second transition is the point $[2, 2]$. Now, we want the sum of both $|x - 2|$ and $\min_{x'-x\in[2,2]} g_1(x')$. The first of these functions, that is the contribution of the last timestamp to the error, is pictured in figure 1 in green dashed lines. The second component is pictured in black dashed lines. In this case, the non-green dashed component is simply a translation of g_1, as $\min_{x'-x\in[2,2]} g_1(x') = g_1 - 2$.

Now, let us construct g_3 similarly, by summing the contribution of the last timestamp, $|x - 4|$, to $\min_{x'-x\in[2,4]} g_2(x')$. We now have an orange segment in the second component! This is used to demarcate any additional segments not present in the previous g-function (if any). We see the effect of the min *operation is simply to attach a segment with the same length as the ith static interval constraint, at the lowest point of g_{i-1}. Now we begin to see a simple way to construct these graphs visually : The next black graph is a translation of the previous blue one, with an orange line segment or the same length as the next static interval constraint inserted at the lowest point, and the next green graph is simply $|x - \sigma_i|$ in the new domain, and summing these two gives the next blue graph. Even summation is simpler than it looks, as these are all piecewise linear graphs, so you can simply increment the slopes of all the segments of the black-orange curve by one if they're to the right of σ_i, and by minus one if they're to the left, and translate the height up as needed. Clearly the orange, black and green segments of the next graph are obtained like this, and g_4 is their sum.*

The procedure for constructing g_5 and g_6 are identical, and now finally we see that the minimum cost of 2 is achieved in g_6 by setting the last timestamp to 8. Now, we know the contribution of the 8 to the error is zero, so this means we can deduce the corresponding value in g_5 must have been 2 as well, achieved when the fifth timestamp was set to 6. Now 6 gives a contribution of 0 again to the error, letting us look at the preimage of 2 once again on g_4, and so on. We can backtrack in this manner to build an optimal alignment, such as in this case, $(0, 2, 5, 6, 6, 8)$.

With this context, we present the following algorithm

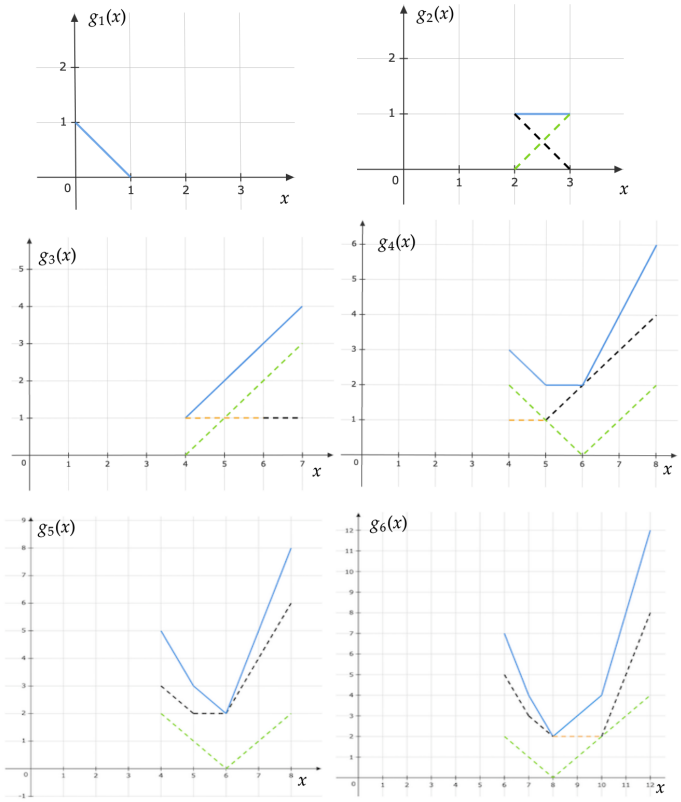


Fig. 1. Visual representation of a run of the algorithm on Example 8

that progressively calculates the graphs of g_i each round, and claim it is both correct and efficient.

The algorithm does precisely the procedure delineated above, where the subfunction GRAPHMIN takes $g_i(d)$ and outputs

$$\min_{d-d'\in[a,b]} g(d') = \begin{cases} g(d-a) & l + a \le d < m + a \\ g(m) & m + a \le d \le m' + b \\ g(d+b) & m' + b < d \le h + b \end{cases}$$

Which is exactly the process of taking the last blue graph and creating the next orange-black graph, and subfunction GRAPHADDMOD takes $\min_{d-d'\in[a,b]} g(d')$ and adds $|d - \sigma_{i+1}|$ to it, which is precisely adding the green component to get the next blue graph.

Theorem 5. *Algorithm 2 is correct, that is, given a linear causal process of a time Petri Net N, its result γ has the properties :*

1) $\gamma \in \mathcal{L}(N)$
2) $\forall x \in \mathcal{L}(N) : d_t(\gamma, \sigma) \le d_t(x, \sigma)$

Remark 3. *Note that the subfunctions* GRAPHMIN *and* GRAPHADDMOD *both run through the list of segments of the graph once each, and hence are linear in the sizes of their inputs, and Algorithm 2 calls each of these subfunctions once for each letter of the observed trace, each*

Algorithm 2 d_t Algorithm

Input : σ, a list of static interval constraints for the linear model SI
Output : $cost = \min_{x \in \mathcal{L}(N)} d_t(x, \sigma)$, $\gamma \in \mathcal{L}(N)$ such that $d_t(\gamma, \sigma) = cost$

Object : $graph = \{(left, slope, right) | graph[i].left = graph[i-1].right\}$
procedure StampOnlyAlgo(σ, SI)
  Initialise System
  $a \leftarrow \pi_1(SI[0])$
  $b \leftarrow \pi_2(SI[0])$
  $graph \leftarrow \{(a, 0, b)\}$
  graphAddMod($graph, \sigma[0]$)
  $graphlist \leftarrow \{graph\}$
  $i \leftarrow 1$
  $cost \leftarrow |a - \sigma[0]|$
  while $i < |sigma|$ **do**
    $a \leftarrow \pi_1(SI[i])$
    $b \leftarrow \pi_2(SI[i])$
    graphMin($graph, a, b$)
    graphAddMod($graph, \sigma[i]$)
    $graphlist.append(graph)$
    $cost \leftarrow cost + |a - \sigma[i]|$
  end while
  $i \leftarrow 0$
  $s \leftarrow graph[0].slope$
  while $graph[i].slope < 0$ **do**
    $s \leftarrow graph[i].slope$
    $cost \leftarrow cost + s * (graph[i].right - graph[i].left)$
    $i \leftarrow i + 1$
  end while
  $\gamma \leftarrow$ backTrack($\sigma, graphlist, cost, SI$)
  Return : $cost, \gamma$
end procedure

time on a graph of the size of the current prefix. So the cost calculation section of Algorithm 2 has quadratic time complexity in the size of the input.

The backtracking subfunction backTrack *runs a binary search on each stored graph in graphlist to find a trace in the language that does indeed give the minimum cost, letter by letter, and so overall takes $O(nlogn)$ time, keeping the overall time complexity of Algorithm 2 $O(n^2)$ where $n = |\sigma|$.*

VI. The General Timed Alignment Problem

Now that we have provided some ways to tackle the purely timed alignment problem in a number of metrics and settings, we come back to the general timed alignment problem. One simple approach for solving the general timed alignment problem would be simply to ignore the timed aspect completely at the start, viewing only the

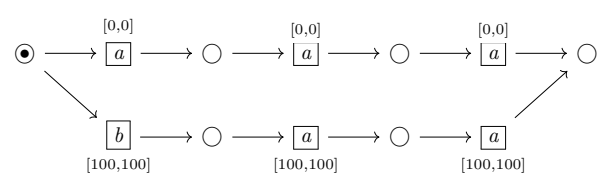
Algorithm 3 Backtrack to find the best word

Input : σ, $graphlist$, $cost$, SI
Output : $\gamma \in \mathcal{L}(N)$ such that $d_t(\gamma, \sigma) = cost$
for $i = 0, i < |graphlist|$ **do**
  Find $ylist[i]$, the y values of $graphlist[i]$
end for
$i \leftarrow |sigma| - 1$
while $i \geq 0$ **do**
  $[a, b] = SI[i+1]$
  Truncate $ylist[i]$ and $graphlist[i]$ so $\gamma_{i+1} - x \in [a, b]$.
  $k \leftarrow$ binarySearch($ylist[i], cost$)
  $xsegment \leftarrow graphlist[i][k]$
  $\gamma_i \leftarrow xsegment^{-1}(cost)$
  $cost \leftarrow cost - |\gamma_i - \sigma_i|$
  $i \leftarrow i - 1$
end while

untimed part of the word w and the process model N (now reduced to the underlying untimed Petri net) and applying the A^* algorithm, or any other untimed alignment approach. This will produce an untimed word in the language of the untimed model, that is, a word w' with a valid causal process. This new untimed word w' can then be given a timing function based on that of w, preserving time for substitution edits, synchronous moves, and repeating or deleting timestamps for any pure insertion or deletion moves. This now leaves us with a valid causal process and a potentially invalid timing function over it, which can then be cast as a purely timed alignment problem.

This approach is quite naive, as it may have a tendency to exaggerate the importance of action label errors. For example, in the following scenario, perhaps the second alignment is better than the first, but it would be overlooked by the approach we described above.

Example 9. *Consider the following net N_3 where the final marking is presumed to be the sink place:*



Now, let the word we seek to align be $w = (a, 100)(a, 100)(a, 100)$.

The alignment the approach we described above would provide is $w' = (a, 0)(a, 0)(a, 0)$.

But if we assign even a tenth of the cost of action edits to timestamp edits, clearly the closer word in the language was $w'' = (b, 100)(a, 100)(a, 100)$.

This leaves us once again with the question of how to

balance the tradeoff between timestamp edits and action label edits. A more balanced approach would assign a particular cost c_A to action edits and another c_T to time edits, and devise an algorithm to minimise their sum. A potential fix to our previous idea would be to essentially retain the above approach, but instead of just doing it for the best untimed alignment, take a selection of good untimed alignments (up to some allowable threshold), and try to align their timestamps for each untimed candidate, and then minimise the total cost over this set. This alleviates some of the bias towards preserving action labels by allowing for action deviations in case the timestamp alignment proves particularly easy, but this is still just an initial approach to the wider problem, and we hope to find better methods in the future.

VII. Implementation

We have implemented the stamp only alignment algorithm in python, available at <link>. This algorithm is quadratic in time complexity, and runs well in practice, as seen below :

VIII. Perspectives and Conclusion

References

[1] W. M. P. van der Aalst, *Process Mining - Data Science in Action, Second Edition.* Springer, 2016. [Online]. Available: https://doi.org/10.1007/978-3-662-49851-4

[2] J. Carmona, B. F. van Dongen, A. Solti, and M. Weidlich, *Conformance Checking - Relating Processes and Models.* Springer, 2018. [Online]. Available: https://doi.org/10.1007/978-3-319-99414-7

[3] A. Adriansyah, "Aligning observed and modeled behavior," 2014.

[4] M. Boltenhagen, T. Chatain, and J. Carmona, "A discounted cost function for fast alignments of business processes," in *Business Process Management - 19th International Conference, BPM 2021, Rome, Italy, September 06-10, 2021, Proceedings*, ser. Lecture Notes in Computer Science, A. Polyvyanyy, M. T. Wynn, A. V. Looy, and M. Reichert, Eds., vol. 12875. Springer, 2021, pp. 252–269. [Online]. Available: https://doi.org/10.1007/978-3-030-85469-0_17

[5] S. Cheikhrouhou, S. Kallel, N. Guermouche, and M. Jmaiel, "The temporal perspective in business process modeling: a survey and research challenges," *Service Oriented Computing and Applications*, vol. 9, pp. 75–85, 2014.

[6] J. Eder, E. Panagos, and M. Rabinovich, "Time constraints in workflow systems," in *CAiSE*, 1999.

[7] A. Nguyen, S. Chatterjee, S. Weinzierl, L. Schwinn, M. Matzner, and B. Eskofier, *Time Matters: Time-Aware LSTMs for Predictive Business Process Monitoring*, 03 2021, pp. 112–123.

[8] W. Aalst, H. Schonenberg, and M. Song, "Time prediction based on process mining," *Inf. Syst.*, vol. 36, pp. 450–475, 04 2011.

[9] A. Rogge-Solti, R. Mans, W. M. P. van der Aalst, and M. Weske, "Repairing event logs using timed process models," in *On the Move to Meaningful Internet Systems: OTM 2013 Workshops - Confederated International Workshops: OTM Academy, OTM Industry Case Studies Program, ACM, EI2N, ISDE, META4eS, ORM, SeDeS, SINCOM, SMS, and SOMOCO 2013, Graz, Austria, September 9 - 13, 2013, Proceedings*, ser. Lecture Notes in Computer Science, Y. T. Demey and H. Panetto, Eds., vol. 8186. Springer, 2013, pp. 705–708. [Online]. Available: https://doi.org/10.1007/978-3-642-41033-8_89

[10] R. Conforti, M. L. Rosa, A. H. M. ter Hofstede, and A. Augusto, "Automatic repair of same-timestamp errors in business process event logs," in *Business Process Management - 18th International Conference, BPM 2020, Seville, Spain, September 13-18, 2020, Proceedings*, ser. Lecture Notes in Computer Science, D. Fahland, C. Ghidini, J. Becker, and M. Dumas, Eds., vol. 12168. Springer, 2020, pp. 327–345. [Online]. Available: https://doi.org/10.1007/978-3-030-58666-9_19

[11] W. M. P. van der Aalst and L. F. R. Santos, "May I take your order? - on the interplay between time and order in process mining," in *Business Process Management Workshops - BPM 2021 International Workshops, Rome, Italy, September 6-10, 2021, Revised Selected Papers*, ser. Lecture Notes in Business Information Processing, A. Marrella and B. Weber, Eds., vol. 436. Springer, 2021, pp. 99–110. [Online]. Available: https://doi.org/10.1007/978-3-030-94343-1_8

[12] T. Aura and J. Lilius, "Time processes for time petri-nets," in *Application and Theory of Petri Nets 1997, 18th International Conference, ICATPN '97, Toulouse, France, June 23-27, 1997, Proceedings*, ser. Lecture Notes in Computer Science, P. Azéma and G. Balbo, Eds., vol. 1248. Springer, 1997, pp. 136–155. [Online]. Available: https://doi.org/10.1007/3-540-63139-9_34

[13] B. Barthomieu and M. Menasche, "A state enumeration approach for analyzing time petri nets," 1982. [Online]. Available: https://projects.laas.fr/tina/papers/atpn82.pdf

[14] S. P. Boyd and L. Vandenberghe, *Convex Optimization.* Cambridge University Press, 2014. [Online]. Available: https://web.stanford.edu/%7Eboyd/cvxbook/

Appendix

A. Delay Only

1) Timed Processes: Now, in order to study *timed* executions, we want to be able add a timing function to the causal process we defined above, thereby allowing us to record when different transitions are taken. The following definitions, properties and theorems in this subsection cover the material developed in Aura and Lilius' article on Time Processes [12].

Given the definition of the timing function, we try to see how a time process unfolds. First the initial events, that is, those enabled at $Min(CN)$ happen, and as each event occurs, new events are enabled and disabled. In order to keep track of this, we first define the auxiliary Cut function that represents the effect of having all the events in a subset fire simultaneously (we only use Cut on sets of events that can concurrently fire), as defined below.

$$Cut(E') = (E'^{\bullet} \cup Min(NS)) \setminus {}^{\bullet}E'.$$

Now we can define the *time of enabling* for a transition t of a TPN in a set of conditions B' of its causal process as $TOE(B', t) =$

$$max(\{\tau({}^{\bullet}b) | b \in B' \setminus Min(CN) \wedge p(b) \in {}^{\bullet}t\} \cup \{0\})$$

We must now verify that this timing function does indeed represent a valid execution that obeys all the static interval constraints, as below :

Definition 15 (Valid Timing)**.** *A timing function τ is a valid timing of the causal process iff*

$$\forall e \in E : \tau(e) \geq TOE({}^{\bullet}e, p(e)) + Eft(p(e))$$

$$\forall e \in E : \forall t \in Enabled(p(C_e)) : \tau(e) \leq TOE(C_e, t) + Lft(t)$$

where $C_e = Cut(Earlier(e))$.

Here, checking every element of C_e might seem unnecessary but in certain nets, complex dependencies between transitions can cause causally unrelated transitions to force transitions to fire or be disabled, due to urgency. This phenomenon is known as confusion, and hence to guard against this, a timing function must check that it never leaves a section of the process behind, completing the firing or disabling of every event in $Earlier(e)$ before it can reason about the fireability of e.

Definition 16 (Time Process)**.** *A time process of a TPN is a triple* (CN, p, τ) *where* τ *is a valid timing of* (CN, p) *which is a causal process of the TPN.*

A few last notions here will assist us when we consider a simpler class of time Petri nets, that allow for much easier validity checking.

Definition 17 (Extended Free Choice)**.** *A time Petri net is extended free choice iff for all two transitions* t *and* t', ${}^\bullet t \cap {}^\bullet t' \neq \emptyset$ *implies* ${}^\bullet t = {}^\bullet t'$.

This class of time Petri nets ensure that the net is *confusion-free*, that is, causally unrelated events cannot affect the fireability of other events. This means that in order to check for the validity of a timing function, checking all of C_e as before is no longer necessary.

How do we construct a valid timing function on the fly? Given a partial time process of a TPN, we want to be able to study the effect of firing particular transitions amongst the set of transitions currently enabled. In order to do so, we notice the following class of transitions.

A transition t is a *choice* at $B' \subseteq B$ iff B' is a co-set that maps injectively to places and ${}^\bullet t = p(B')$, and a choice is an *extension* of the process iff $B' \subseteq Cut(E)$.

These transitions reflect exactly the transitions enabled at a particular moment in the evolution of the process. Either they must be fired or disabled, by the urgency condition.

Given the above, Aura and Lilius characterise a method by which one can build partial time processes inductively, building forward while maintaining the validity of the timing function, denoted as keeping the process complete with respect to the timing function :

Definition 18. *A causal process* (CN, p) *of a time Petri Net where* $CN = (B, E, G)$ *is said to be complete with respect to a timing function* τ *iff for every extension transition* t *of the process,*

$$\max\{\tau(e) | e \in E\} \leq TOE(Cut(E), t) + Lft(t).$$

Now given this, the following theorem due to Aura and Lilius [12] holds :

Theorem 6. *Let* N *be an extended free choice time Petri net and* (CN, p) *a causal process of* N*, where* $CN = (B, E, G)$*. A timing function* τ *is valid iff the following criteria hold :*

1) $\forall e \in E : Eft(p(e)) \leq \tau(e) - TOE({}^\bullet e, p(e))$
2) $\tau(e) - TOE({}^\bullet e, p(e)) \leq \min\{Lft(t) | {}^\bullet t = {}^\bullet p(e)\}$
3) (CN, p) *is complete with respect to* τ*, i.e, for every extension transition* t *of the process,*

$$\max\{\tau(e) | e \in E\} \leq TOE(Cut(E), t) + Lft(t)$$

2) Proof of correctness:

Proof. By theorem 6 proved in Aura and Lilius' article [12], we know that in order to ensure that this is a valid time process, we need only ensure three inequalities hold as the time process γ evolves.

1) $\forall e \in E : Eft(p(e)) \leq \gamma(e) - TOE({}^\bullet e, p(e))$
   By definition $f_\gamma(e) = \gamma(e) - TOE({}^\bullet e, p(e))$, and by line 6 of the algorithm the variable eft is correctly initialised to store $Eft(p(e))$, and line 17 ensures the above inequality holds.
2) $\gamma(e) - TOE({}^\bullet e, p(e)) \leq \min\{Lft(t) | {}^\bullet t = {}^\bullet p(e)\}$
   In similar vein, line 6 also ensures that l is initialised precisely to store $\min\{Lft(t) | {}^\bullet t = {}^\bullet p(e)\}$, and line 17 again enforces the left hand side to be less than l.
3) (CN, p) is complete with respect to γ, i.e, for every extension transition t of the process,

$$\max\{\gamma(e) | e \in E\} \leq TOE(Cut(E), t) + Lft(t)$$

   At every point when assigning a timestamp to an event, it is ensured that it belongs to *Soon*, a set defined to contain only those enabled events that have the least value of $l + toe$, or, $TOE(Cut(E), p(e)) + Lft(p(e))$. This ensures that whenever an event e is assigned a timestamp $\gamma(e)$, for all extension transitions t (which must either be causal descendents of e or already enabled when e was picked, thereby being in *Enabled*, by extended free choice) we know that $\gamma(e) \leq TOE(Cut(E), p(e)) + Lft(p(e)) \leq TOE(Cut(E), t) + Lft(t)$.
   Hence, (CN, p, γ) is a valid time process of N.

As for its optimality, we see that any other valid time process would have to obey the same inequalities, in particular for another valid timing function γ' it would have to obey

$$f_{\gamma'}(e) \in [Eft(p(e)), \min_{{}^\bullet t' = {}^\bullet p(e)} Lft(t')] = [a, b]$$

and hence at any event e where γ differs from γ' it would have lower than or equal cost for that event as

$$f_\gamma(e) = \operatorname*{argmin}_{x \in [a,b]} |f_\sigma(e) - x|$$

Hence, the algorithm is correct. □

B. Stamp Only

Proof. Lemma 1 (General Form Lemma) of [13] states that the firing domains of state classes for any bounded time Petri net may be expressed as solution sets of systems of inequalities of the following form :

$$\begin{cases} a_i \leq t(i) \leq b_i & \forall i \\ t(j) - t(k) \leq c_{jk} & \forall j \neq k \end{cases}$$

As $\gamma \in \mathcal{L}(N)$ this means it obeys the set of inequalities corresponding to the state class containing the final marking. Now, the above constraint is convex, so γ varies over a convex set. As σ is a constant vector, translation by it will keep the space convex, so $\gamma - \sigma \in \mathcal{C}$ for some convex set $\mathcal{C}$.

Now, we wish to minimize $|\gamma - \sigma|$, but it would be better to view this objective as a linear function. To this end we use a trick from [14] and use the following variable translation :

$$\alpha, \beta \geq 0 : \alpha - \beta = \gamma - \sigma$$

As defined above, α and β are the absolute values of the nonnegative and negative components of the vector $\gamma - \sigma$, respectively, so their difference is $\gamma - \sigma$ and their sum is $|\gamma - \sigma|$. Clearly now, we have completed the transformation, and our alignment problem is equivalently cast as

$$\underset{\alpha-\beta \in \mathcal{C}, \alpha, \beta \geq 0}{\text{minimize}} \quad \alpha + \beta$$

□

Before we prove Theorem 5, we prove a small, relevant lemma.

Lemma 7. *For any convex piecewise linear function g, the following function g' is also convex and piecewise linear.*

$$g'(d) = \min_{d-d' \in [a,b]} g(d')$$

Proof. For any piecewise linear convex function g, the function

$$g'(d) = \min_{d-d' \in [a,b]} g(d')$$

is also piecewise linear. If the domain of g was $[l, h]$, the domain of g' is $[l + a, h + b]$.

Say the leftmost minimum of g (if g has multiple minima they are all on the same flat line segment) is at $m \in [l, h]$. For all $x \in [l + a, m + a]$ and $x - y \in [a, b]$ and $y \in [l, h]$ implies $y \in [x - b + a, x - a] \cap [l, h] \subseteq [l, h] \cap [l - b + am] = [l, m]$ meaning for each x, in the possible domain of y $g(y)$ is a strictly decreasing function, hence we have

$$\forall x \in [l + a, m + a] : \min_{x-y \in [a,b]} g(y) = g(x - a)$$

Now, let the rightmost minimum of g is m'. For all $x \in [m + a, m' + b]$ and $x - y \in [a, b]$ and $y \in [l, h]$ implies $y \in [x - b, x - a] \cap [l, h]$, and $[x - b, x - a] \cap [m, m'] \neq \emptyset$. Now, this means for each $x \in [m + a, m' + b]$ the domain of y has a minimum of g, so we have

$$\forall x \in [m + a, m' + b] : \min_{x-y \in [a,b]} g(y) = g(m)$$

Lastly, let $x \in [m' + b, h + b]$ and $x - y \in [a, b]$ and $y \in [l, h]$. This implies $y \in [x - b, x - a] \cap [l, h] = [m', h]$, so for each $x \in [m + a, m' + b]$ the domain of y is in a region where g is strictly increasing, so we have

$$\forall x \in [m + a, m' + b] : \min_{x-y \in [a,b]} g(y) = g(x + b)$$

So we see that for any convex piece-wise linear function g,

$$\min_{d-d' \in [a,b]} g(d') = \begin{cases} g(d - a) & l + a \leq d < m + a \\ g(m) & m + a \leq d \leq m' + b \\ g(d + b) & m' + b < d \leq h + b \end{cases}$$

Hence g' is also convex and piecewise linear, and has at most one more linear segment than g. □

We may now prove Theorem 5 as follows :

Proof. We are given a process model N, an observed trace σ, and its underlying linear causal process $CN = (B, E, G)$ with homomorphism p for the untimed run of σ on N. As discussed earlier, for linear process models, we define

$$g_i(d) = \min_{\tau|_i \in \mathcal{L}^i(N) \wedge \tau_i = d} cost(\tau|_i)$$

This family of functions is seen to obey the following recursion:

$$g_{i+1}(d) = |d - \sigma_{i+1}| + \min_{d-d' \in [a,b]} g_i(d')$$

If we can calculate the family of functions $\forall i \leq n : g_i$ efficiently, then the minimal cost for aligning σ to N is simply

$$\min_d g_n(d)$$

Hence to prove the correctness of Theorem 5 we only need show that the auxiliary subfunctions called do indeed do their job correctly, as the main function simply implements the recursion.

To this end, we first notice that the family of functions $g_i(d)$ are convex and piecewise linear. This can be seen through induction :

Firstly $g_1(d) = |d - \sigma_1|$ so the base case is covered.

Now suppose g_{i-1} is known to be convex and piecewise linear for some $i \geq 2$. By Lemma 7 we see that $\min_{d-d' \in [a_i, b_i]} g_{i-1}(d')$ is also convex and piecewise linear, and so is $|d - \sigma_i|$ and both these properties are preserved by addition, so $g_i(d)$ must also be convex and piecewise linear.

Now that we have this result, our representation of the graph as a sequence of line segments, each represented as a triple (left x endpoint, slope, right x endpoint) is justified. The y value of any graph g_i can be back calculated because

we know the value of the cost of the leftmost endpoint, it represents the word obtained by firing every transition at the earliest possible time.

Now subfunction GRAPHADDMOD is adding the point σ_i if it is inside the domain of the function, and changing the slopes of the segments before σ_i by subtracting one, and of those after σ_i by adding one, which has the effect of adding the modulus component $|d - \sigma_i|$ to the previous graph.

Subfunction GRAPHMIN is even simpler, as Lemma 7 suggests all it does is translate the whole graph forward by a, and then translate the strictly increasing portion from $[m', h]$ by an addition $b - a$, adding an extra flat segment of length $b - a$ in between to keep the two portions of the function connected, and so on input $g(d), [a, b]$ it outputs $\min_{d-d' \in [a,b]} g(d')$.

Subfunction BACKTRACK takes the minimum cost obtained by the calculation of the g_n function, finds the value of d, ie γ_n for which the minimum is achieved, and subtracts the cost aligning only the last place incurs, thereby finding the minimum cost for aligning the $n - 1$ length prefix. It then proceeds to do the same iteratively for each prefix, reverse-engineering a trace γ for which the total minimum cost of aligning to σ is achieved.

□